arXiv:2406.02844v2 [cs.IR] 16 May 2025

Item-Language Model for Conversational Recommendation

Li Yang
Google Research
lyliyang@google.com

Anushya Subbiah
Google Research
anushyas@google.com

Hardik Patel
Google
patelhardik@google.com

Judith Yue Li
Google Research
judithyueli@google.com

Yanwei Song
Google
yanweisong@google.com

Reza Mirghaderi
Google
rezam@google.com

Vikram Aggarwal
Google Research
viki@google.com

Qifan Wang
Meta AI
wqfcr@meta.com

Abstract

Large-language Models (LLMs) have been extremely successful at tasks like complex dialogue understanding, reasoning and coding due to their emergent abilities. These emergent abilities have been extended with multi-modality to include image, audio, and video capabilities. Recommender systems, on the other hand, have been critical for information seeking and item discovery needs. Recently, there have been attempts to apply LLMs for recommendations. One difficulty of current attempts is that the underlying LLM is usually not trained on the recommender system data, which largely contains user interaction signals and is often not publicly available. Another difficulty is user interaction signals often have a different pattern from natural language text, and it is currently unclear if the LLM training setup can learn more non-trivial knowledge from interaction signals compared with traditional recommender system methods. Finally, it is difficult to train multiple LLMs for different use-cases, and to retain the original language and reasoning abilities when learning from recommender system data. To address these three limitations, we propose an Item-Language Model (ILM), which is composed of an item encoder to produce text-aligned item representations that encode user interaction signals, and a frozen LLM that can understand those item representations with preserved pretrained knowledge. We conduct extensive experiments which demonstrate both the importance of the language-alignment and of user interaction knowledge in the item encoder.

1 Introduction

Large Language Models (LLMs), trained on web-scale data using a very large number of parameters, have shown remarkable emergent capabilities, such as in-context learning, reasoning, coding [5, 7, 19]. Recently, those abilities have been extended to multimodal domains, including image, audio, and video [39, 16, 15]. On various professional and academic benchmarks, those models achieve or surpass human-level performance [22]. By contrast, the improvement in recommender systems have not derived similar breakthrough with improvements in LLM, even though recommender systems support a large volume of online user activity. One reason is that users' interactions with recommender systems are typically not in natural language form. For example, a video recommender

Preprint. Under review.

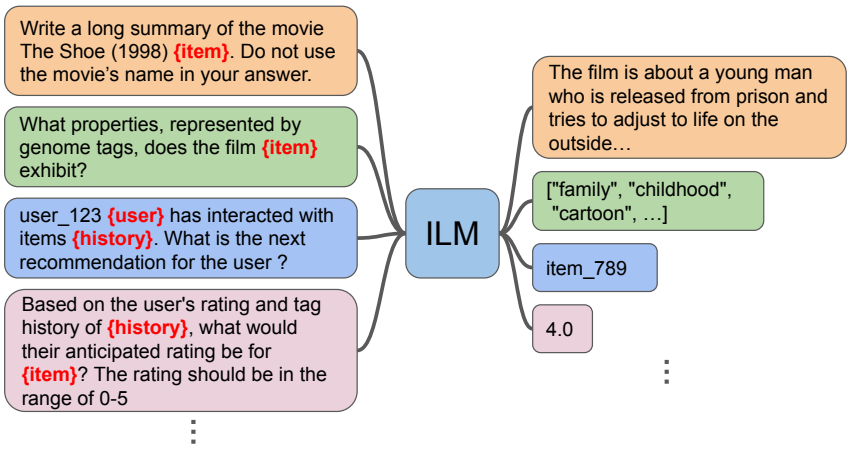

Figure 1: Conversational recommendation tasks using ILM. User and item collaborative filtering embeddings, marked by placeholders in the input, are interleaved with text embeddings and fed to the model. Where {history} is a sequence of items.

system typically recommends videos to users based on their implicit preferences from watch history and other query and candidate features, and users usually do not provide natural language preferences. In order to induce such natural language preferences, conversational recommendations have been proposed [8, 4, 18], where the user can interact with the system explicitly using either natural-language tags or conversational natural language. With the development of techniques such as LLM tool-use [45, 41], Retrieval Augmented Generation [32, 14], and Agents [40, 52], LLMs have demonstrated breakthroughs in many domains. It is possible that those technical advances and the resulting user habit changes will lead to conversational recommender systems to be more important in the future, thus it is worthwhile to develop methods to bring these benefits to conversational recommender systems.

However, using an LLM for conversational recommender systems has the following difficulties. (1) Unlike multimodality, to the best of our knowledge no current LLM natively understands both user interaction signals and natural language. Current LLMs are trained on natural language data or natural language aligned contents, e.g. image with description, audio with transcription, etc. User interaction signals exist in the form of item co-watch, for example, if many users have viewed both v_1 and v_2, then a user who likes v_1 may also like v_2. Those signals often have a complicated nature, e.g. it is hard to explain why people tend to watch v_1 and v_2 together, so there is often a lack of natural language descriptions of such signals. Due to this, existing approaches using In-Context Learning [5] by prompting LLMs for recommendation tasks purely using natural language struggle to achieve superior performance [9, 13, 28, 58, 51]. Other approaches finetuning LLMs on user interaction signals [3, 28] show improvement upon purely prompting based methods. However, after such finetuning, the LLM's original abilities are often forgotten or hidden, and new privacy concerns may arise if user data are not handled correctly [25]. (2) Although it is possible to annotate an item with a text format context of all related items/users, or a user with a text format context of all related items, by doing so, the context for the user/item could be very long, which will dramatically increase the inference cost. (3) Finally, another way to incorporate behavioral data is to first compute item and user collaborative filtering embeddings using matrix factorization algorithms, then feed those embeddings to LLM through a mapping module [49, 59]. This approach introduces a modality gap between collaborative embeddings and LLM pretrain token embeddings, so the LLM still needs to be finetuned to close this modality gap.

Towards solving the above difficulties, we propose Item-Language Model (ILM, hereafter) for conversational recommendation tasks, Figure 1. ILM is a two-phase framework containing an item-language representation learning phase and an item-language model training phase. The overall model architecture is shown in Figure 2(d). Motivated by the BLIP-2 [33] work of bridging the modality gap with a lightweight Querying Transformer (Q-Former), we adopt Q-Former to generate item-language aligned representations. In Phase 1, we pretrain a Q-Former encoder following the BLIP-2 approach. In addition to the original item-text objectives, we also introduce an item-item

contrastive objective, which plays a regularization role and also encodes co-watch information in the resulting item-language representation. The inputs to the item encoder are collaborative filtering embeddings. Throughout the paper, we treat the user as a special item. In Phase 2, we integrate the Q-Former into a pretrained LLM through a linear projection adaptor layer, and finetune the ILM on conversational recommendation tasks in multitasking fashion. During finetuning, only the Q-Former and the adaptor parameters are updated, the pretrained LLM is kept frozen to preserve its pretrained abilities. The main contributions of this work are summarized as follows:

1. We propose ILM, a novel framework for using a Q-Former item encoder to produce item-language aligned representations from collaborative filtering embeddings, then integrate into a frozen LLM for conversation recommendation tasks with interleaved item-text inputs.
2. We conduct extensive benchmarks and ablation studies on various conversation recommendation tasks to show that our ILM approach consistently outperforms existing approaches of integrating item representations into LLMs across all tasks.

Paper Overview. In Section 2, we provide a brief literature survey of LLM for recommender systems, how item representations are used there and multimodality LLM methods. In Section 3, we present the details of the proposed ILM approach, including model architecture and training. We present our experiment results in Section 4 and ablation studies Section 5. We conclude in Section 6.

2 Related Work

LLM for recommendation With LLMs showing remarkable emergent abilities and surpassing human-level performance across various domains, there have been explorations to apply LLMs to recommender systems [36, 54]. In-Context Learning methods have been used as a straightforward way for this purpose [9, 13, 28, 58, 51], which rely on LLMs' world knowledge. However, since user interaction data in recommender systems is largely not available during LLM pretraining, purely text-prompting based methods show suboptimal performance. Another line of work is by finetuning a language model on user interaction data. P5 [17, 55] pretrains a unified language model for many different recommendation tasks by converting them into a common natural language sequence format, where user and item ids are represented as text strings. TALLRec [3] finetunes a LLM using LoRA [23] on user rating data, and the model outputs a binary label.

Item representations in LLM4Rec Efficiently representing users and items in recommender systems is a rich field with years of work of traditional techniques such as Matrix Factorization [30, 44]. When applying LLM to recommender systems, users and items are key objects, and it is critical for LLM to be able to understand them. Using text representation, such as the title of an item is a straightforward way. However, one issue is text representation of an item may not be informative enough. For example, it is quite common that a video title is unrelated to the video content, and different content can have a single title. We can always include more text features of an item, such as description, author and other content features, but this may introduce irrelevant information, which may confuse the model or make the method inefficient due to very long input. Quantized id representations as item representation has been proposed in the methods of random indexing [2, 55], sequential or collaborative indexing [55, 26] and semantic ids in the context of generative retrieval [47, 43, 48]. In addition to using quantized ids, ELM [49] demystifies the input embedding spaces by feeding semantic embeddings to LLM, and CoLLM [59] enhances recommendation performance on rating prediction tasks by feeding user and item collaborative filtering embeddings to LLM. Recently, USER-LLM [38] contextualizes LLM with user history embeddings by integrating user embeddings to LLM through perceiver [27, 1], projection, and cross-attention modules, where the LLM can be frozen to preserve the original ability.

Multimodal LLM Our approach is closely related to a series of multimodal foundation models trained with both generative and contrastive learning objectives, including BLIP-2 [33], CoCa [56], and MaMMUT [31], which achieve state-of-art performance on vision-language tasks. In our case the pretrained LLM is kept frozen, requiring a pretrained adaptor with enough representation capacity like Q-Former [33] used in BLIP-2 and GILLMapper [29]. Compared to CogVLM [53], which introduces a visual module to modify the frozen LLM directly, our approach is considered as late fusion. Our two-phase workflow mostly resemble BLIP-2's two-phase workflow, including pretraining an adapter

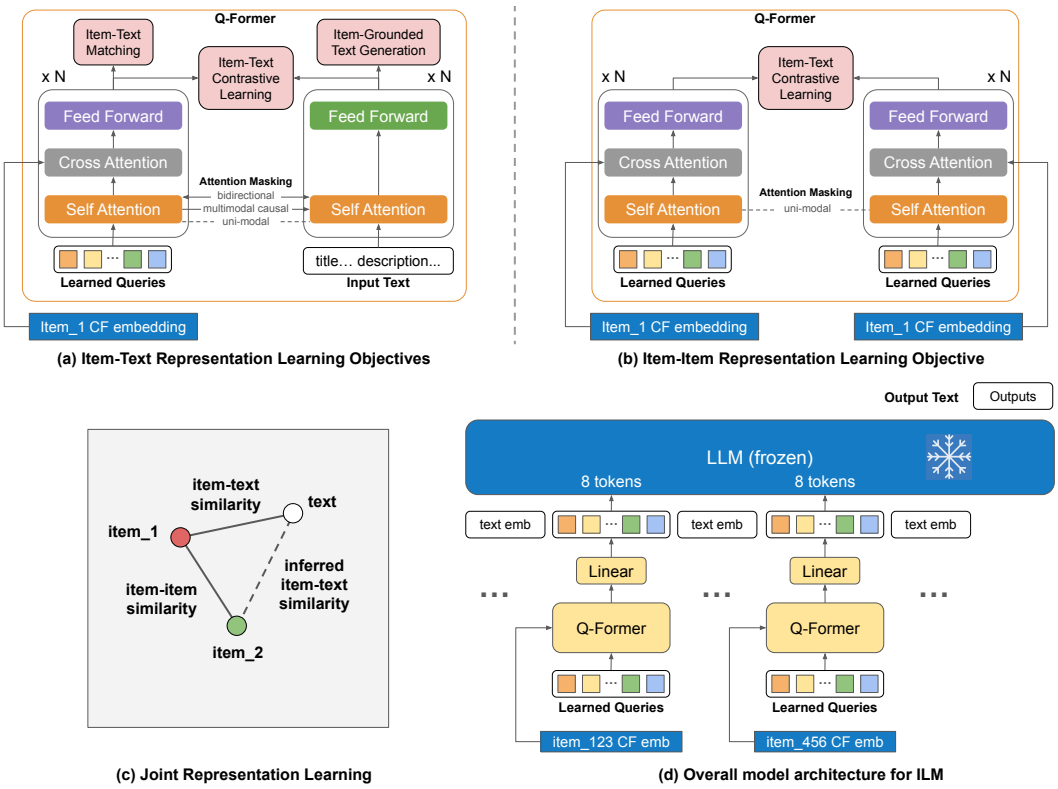

Figure 2: Overall model architecture for ILM. **(a)** The original item-text contrastive, item-grounded text generation and item-text matching losses used in BLIP-2 [33] in Q-Former phase 1 training. **(b)** The new item-item contrastive loss we introduced in Q-Former phase 1 training. For user-item contrastive learning, we simply replace item collaborative filtering (CF) embedding with user CF embedding. **(c)** A schematic of how item-item contrastive learning can improve text-aligned item representations. **(d)** The ILM phase 2 training by integrating the Q-Former to a frozen LLM.

in phase one and task fine-tuning in phase two. Compared to general purpose vision-language model [6, 57], our approach introduces a novel item-item contrastive loss and deals with interleaved item and text inputs [10]. Our approach of keeping LLM frozen and only finetune the item encoder and adaptor is related to Frozen [50], which exhibits great multimodal few-shot learning capability.

3 Method

3.1 Model Architecture

For conversational recommendation tasks, the input and output of the model are typically sequences of interleaved items and text. In addition, for recommendation tasks, the model's ability of understanding items is critical. In this work, we focus on the input item representation with the overall ILM model architecture shown in Figure 2(d). Collaborative filtering information is widely believed to be very important for recommender system performance, and forms the basis of many existing recommender systems. Existing work includes feeding item and user's collaborative filtering embeddings into LLM through a two layer MLP [59]. As can be seen in Figure 2 we adopt the Q-Former [33] architecture to encode items, combined with an item-text alignment pretraining phase to allow the Q-Former to be able to produce text-aligned item representations that also encode collaborative filtering information. We show significant performance gains of this approach on various conversational recommendation tasks in Section 4. The two phases of our proposed ILM model training are discussed in Section 3.2 and Section 3.3, respectively.

3.2 Item-Language Representation Learning

In phase 1, we pretrain the Q-Former item encoder to ensure that it can generate text-aligned item representations given item collaborative filtering embeddings. We adopt the original 3 tasks of item-text contrastive learning, item-text generation and item-text matching in BLIP-2 [33], Figure 2(a). For item-grounded text generation, we use auto-regressive loss on top of the text tower. For item-text matching, we use a binary cross-entropy loss on the CLS token output from the text tower. For item-text contrastive learning, we compute the loss as follows. Given a positive item-text pair, we first compute output representations $[h_1, h_2, ..., h_N]$ of the N learnable queries $[q_1, q_2, ..., q_N]$ from the query tower f_q and use the CLS output representation h_{cls} from the text tower f_t as the text representation:

$$\begin{aligned} [h_1, h_2, ..., h_N] &= f_q([q_1, q_2, ..., q_N], e) \\ h_{\text{cls}} &= f_t([t_{\text{cls}}, t_1, ..., t_L]), \end{aligned} \tag{1}$$

where e is the item input embedding and $[t_1, ..., t_L]$ are text tokens. We select the closest query representation to h_{cls} as the item representation:

$$\begin{aligned} i = \text{argmax}_j \ \ \text{cosine_similarity}(h_j, h_{\text{cls}}) \\ h_{\text{item}} = h_i \end{aligned} \tag{2}$$

Then we compute the contrastive loss between h_{item} and h_{cls} using in-batch negative sampling following [42]. The item-text pair data we used is processed from item descriptions and tags, more details can be found in Section 4.1. In addition to item-text alignment tasks, we also introduce a novel item-item contrastive learning loss to prevent the model from overfitting on item-text data when text labels are sparse, Figure 2(b). The item-item contrastive loss is computed in a similar way by extending the item-text contrastive loss, Equation 1 and Equation 2. Given a positive item-item pair, we first compute output representations of the N learnable queries for both items:

$$\begin{aligned} [h_1^{[1]}, h_2^{[1]}, ..., h_N^{[1]}] &= f_q([q_1, q_2, ..., q_N], e^{[1]}) \\ [h_1^{[2]}, h_2^{[2]}, ..., h_N^{[2]}] &= f_q([q_1, q_2, ..., q_N], e^{[2]}), \end{aligned} \tag{3}$$

where $e^{[1]}$ and $e^{[2]}$ are embeddings for item1 and item2, respectively. Note that we use a single set of learnable queries for all items. We select the pair of closest query representations as the representations for the two items:

$$\begin{aligned} k, l = \text{argmax}_{i,j} \ \ \text{cosine_similarity}(h_i, h_j) \\ h_{\text{item1}} = h_k; \ \ h_{\text{item2}} = h_l \end{aligned} \tag{4}$$

Then we compute the contrastive loss between h_{item1} and h_{item2} in the same way as item-text contrastive loss. Introducing the item-item contrastive loss can provide an additional benefit, namely that the Q-Former output is a better representation. This representation not only encodes item-text similarity knowledge but also encodes item-item similarity knowledge, so that for items without any text labels, it can encode certain text information indirectly, as depicted in Figure 2(c). Due to the in-batch negative sampling of contrastive loss, we cannot mix item-text and item-item examples in the same batch. To address this, we train Q-Former on item-text and item-item batches in an alternating manner. More results on the effects of the item-item contrastive loss are shown in Section 5.1.

3.3 Item-Language Model Training

In phase 2, we integrate the Q-Former item encoder with a pretrained LLM. The LLM can be pretrained for any purpose, which could be a general purpose human instruction followed LLM [39, 16, 15] or a LLM trained on recommender system data that supports generative retrieval of candidate items [43, 48]. We present experimental results for the former in Section 4.3 and the latter in Section 4.4. During training, we freeze the LLM parameters and only tune the parameters in the Q-Former encoder and the linear projection layer. The linear projection layer maps the Q-Former output dimension to the LLM input dimension as shown in Figure 2(d).

4 Experiments

4.1 Datasets

A conversational recommendation can consist of multiple sub-tasks, including user preference elicitation, recommendation, explanation, and item information search [12]. To benchmark our proposed ILM approach on all these tasks, we select the Embedding Language Model (ELM) dataset [49] and the OpenP5 benchmark. The ELM dataset contains 24 tasks which covers user preference elicitation, explanation, and item information search, etc. The OpenP5 dataset contains the sequential recommendation task and the straightforward recommendation task.

ELM 24 Tasks The ELM 24 tasks [49] are created from the MovieLens 25M dataset, including single movie tasks, e.g. summarizing a movie, and movie pair tasks, e.g. comparing characteristics of movies. The training targets are generated by prompting the PaLM 2-L model with a movie's title and task-specific information. For training inputs, the same task-specific prompts are used, but with the movie title replaced by the movie embedding. We use the behavioral embedding in the ELM dataset, which are trained on user ratings in the MovieLens 25M dataset with Matrix Factorization computed using Weighted Alternating Least Squares (WALS) [24].

OpenP5 OpenP5 [55, 26] is a dataset for LLM-based Recommendation development, finetuning, and evaluation. It provides 10 popular preprocessed public datasets, and each dataset contains two kinds of tasks: Sequential Recommendation and Straightforward Recommendation. We select the MovieLens-1M, Beauty, and Clothing datasets with random indexing item representation for our benchmarks, which can be thought of as a simple generative-retrieval setup. The training target for each example is the ground truth item id. For training inputs, we append each item's random indexing id with its behavioral embedding, which are computed using the iALS matrix factorization algorithm [44] on the user sequence training set. We use the provided train, development, and test split in the OpenP5 dataset, which uses the last item in the user sequence for testing and the second from the last item in the user sequence for development.

4.2 Evaluation Metrics

For ELM 24 tasks, we report the log perplexity and Semantic Consistency (SC) [49] on the test set. For SC, we use the cosine similarity of semantic embeddings of the original and decode targets from the Sentence-T5 11B model [37]. For OpenP5 tasks, we report top-k Hit Rate (HR@K) and Normalized Discounted Cumulative Gain (NDCG@K) with K = 5, 10 to evaluate the recommendation performance. To compute those metrics, we use beam search to generate 10 outputs for each example, and remove invalid outputs that do not match the regular expression `".*item_(\d+)$"`.

4.3 Results on ELM 24 tasks

We show the results of our ILM approach on ELM 24 tasks in Table 1. We adopt the CoLLM [59] approach as a baseline, where a two layer MLP with intermediate size 10 times the input embedding size is used to map the input behavioral embedding to LLM token embedding space. For ILM, we use a Q-Former encoder with 8 transformer layers. For phase 1 training, we pair the item with a concatenation of the prompt to generate the original target, i.e. title(s) and task specific information, and the original target as the item-text pair data. For the ELM 24 tasks benchmark, we only train the Q-Former using item-text data to avoid data leakage from item-item data. For all models, we use PaLM 2-S [19] as the LLM backbone. We train the models for 100k steps using a batch size 32 and learning rate 5×10^{-4} with a cosine decay. We also add an **ILM-rand** benchmark, which is an ILM model with a randomly initialized Q-Former encoder. As can be seen, our approach consistently outperforms the MLP baseline across all tasks. **ILM** also outperforms **ILM-rand** by a noticeable margin, which suggests the importance of the item-language representation learning phase. We report mean and standard error of our ILM results in Table 6.

4.4 Results on OpenP5 tasks

In the OpenP5 dataset, each task contains 10 prompt templates used for training, and 1 prompt template used for unseen testing. In Table 4.4, we show seen and unseen test results on OpenP5

Table 1: SC metrics on the ELM 24 tasks using item behavioral embedding. We define SC as the semantic embedding cosine similarity between the decoded text and original text. We adopt the Sentence-T5 11B model [37] for computing semantic embeddings.

Tasks	MLP		ILM-rand		ILM	
	SC(%)	Log pplx	SC(%)	Log pplx	SC(%)	Log pplx
summary	71.68	0.6549	72.69	0.6257	73.88	0.6223
positive review	76.42	0.6114	77.84	0.5708	78.81	0.5669
neutral review	74.61	0.6167	75.98	0.5770	77.94	0.5690
five pos char.	84.96	0.6778	85.74	0.5939	86.92	0.5850
five neg char.	90.77	0.4953	92.46	0.4009	92.24	0.3874
long description	71.09	0.6921	71.98	0.6629	73.39	0.6596
funnier	69.56	0.5744	68.79	0.5401	69.62	0.5378
sadder	70.57	0.5470	71.67	0.5163	72.31	0.5130
scarier	71.55	0.5578	72.07	0.5258	72.90	0.5230
improve	76.37	0.5607	77.11	0.5111	78.15	0.5073
movie to viewer	76.93	0.6801	79.33	0.6041	80.70	0.5920
pitch	78.95	0.5874	83.26	0.5165	84.28	0.5142
criticize	76.62	0.6073	78.13	0.5514	79.63	0.5423
convince1	77.16	0.6859	78.55	0.6196	79.48	0.6106
convince2	78.48	0.8748	78.98	0.7842	80.53	0.7688
convince3	78.02	0.9860	79.01	0.8955	80.33	0.8761
dissuade1	77.90	0.6998	79.00	0.6107	79.49	0.6002
dissuade2	80.97	0.8523	83.71	0.7296	84.15	0.7159
similarities	80.32	0.4993	83.24	0.4082	85.33	0.3828
interpolation	71.43	0.5627	72.59	0.5332	73.70	0.5263
why like nn	78.47	0.7494	79.59	0.6536	80.96	0.6343
diff than nn	86.56	0.6127	88.60	0.5238	89.30	0.4923
common with nn	80.76	0.7008	83.70	0.5466	84.62	0.5234
all	77.48	0.6520	78.96	0.5837	80.01	0.5730
	-	-	+1.91%	-10.47%	+3.27%	-12.12%

MovieLens-1M, Beauty, and Clothing tasks. For **ILM**, we use a Q-Former encoder with 8 transformer layers.

For phase 1 training, we generate the item-text pair data by extracting item metadata from (1) movie title and genres from the original ML1M dataset [20] for the ML1M task (2) product metadata including title, description, features, brand, etc. from the original Amazon Review 2014 Metadata [21, 35] for Beauty and Clothing tasks. Since the inputs of OpenP5 tasks contain both user id and item id, we generate user-item pair data using the training set of OpenP5's user sequence data, and conduct user-item contrastive learning to phase 1 training.

For phase 2 training, we use an 8 layer transformer model as the LLM backbone, and pretrain the backbone on the purely text format OpenP5 data using random item indexing to enable the model generative retrieval ability. We believe our ILM approach can be integrated with any other kind of item token id based encoding such as sequential indexing and collaborative indexing in the OpenP5 dataset as well as other more advanced semantic id based methods [43, 48]. Training hyperparameters can be found in Section B. We show the statistics of phase 1 and phase 2 data in Table 2.

We use the following baselines for the OpenP5 benchmark

- **OpenP5-R** stands for the OpenP5 random indexing method, i.e. using the backbone model directly without any embedding inputs.
- **MLP** stands for using a two layer MLP with hidden size 10 times the input embedding size to map the input behavioral embedding to LLM token embedding space following CoLLM [59]. However, during the training, we do not finetune the LLM backbone in order to preserve the pretrained abilities.

Table 2: OpenP5 phase 1 and phase 2 dataset statistics.

Datasets	Phase 1			Phase 2			
	Item-text	Item-item	User-item	Train	Test	# Users	# Items
ML1M	3079	479664	888696	19629820	12080	6040	3416
Beauty	10879	103268	138521	2628260	44726	22363	12101
Clothing	20750	142427	180128	3210280	78774	39387	23033

Table 3: Results on OpenP5 tasks using item behavioral embedding.

Methods	ML1M				Beauty				Clothing			
	HR@5	NDCG@5	HR@10	NDCG@10	HR@5	NDCG@5	HR@10	NDCG@10	HR@5	NDCG@5	HR@10	NDCG@10
OpenP5-R(seen)	0.0688	0.0455	0.1033	0.0566	0.0208	0.0162	0.0254	0.0177	0.0015	0.0010	0.0030	0.0015
MLP(seen)	0.0692	0.0459	0.1041	0.0572	0.0199	0.0150	0.0255	0.0168	0.0015	0.0010	0.0022	0.0012
ILM-rand(seen)	0.0723	0.0478	**0.1084**	0.0594	0.0206	0.0154	0.0259	0.0171	0.0026	0.0017	0.0038	0.0021
ILM(seen)	**0.0724**	**0.0485**	0.1064	**0.0595**	**0.0213**	**0.0164**	**0.0270**	**0.0182**	**0.0041**	**0.0025**	**0.0065**	**0.0033**
OpenP5-R(unseen)	0.0696	0.0449	0.1041	0.0560	0.0206	0.0161	0.0253	0.0176	0.0016	0.0010	0.0034	0.0016
MLP(unseen)	0.0716	0.0470	0.1045	0.0576	0.0203	0.0151	0.0255	0.0168	0.0016	0.0011	0.0026	0.0014
ILM-rand(unseen)	0.0710	0.0477	0.1033	0.0581	0.0206	0.0157	0.0263	0.0175	0.0023	0.0015	0.0037	0.0019
ILM(unseen)	**0.0717**	**0.0481**	**0.1086**	**0.0600**	**0.0213**	**0.0162**	**0.0269**	**0.0181**	**0.0038**	**0.0024**	**0.0062**	**0.0032**

Table 4: Effects of phase 1 item-item and user-item contrastive losses on OpenP5 benchmarks.

Methods	ML1M				Beauty				Clothing			
	HR@5	NDCG@5	HR@10	NDCG@10	HR@5	NDCG@5	HR@10	NDCG@10	HR@5	NDCG@5	HR@10	NDCG@10
ILM-IT(seen)	0.0719	0.0474	0.1088	0.0594	0.0212	0.0160	0.0262	0.0177	**0.0044**	**0.0029**	0.0061	**0.0035**
ILM-IT-II(seen)	0.0712	0.0479	**0.1093**	**0.0602**	0.0210	0.0160	0.0261	0.0177	0.0040	0.0027	0.0060	0.0033
ILM-IT-UI(seen)	**0.0724**	**0.0485**	0.1064	0.0595	**0.0213**	**0.0164**	**0.0270**	**0.0182**	0.0041	0.0025	**0.0065**	0.0033
ILM-IT(unseen)	0.0700	0.0470	0.1071	0.0589	**0.0218**	**0.0163**	**0.0275**	**0.0182**	**0.0039**	**0.0025**	0.0056	0.0031
ILM-IT-II(unseen)	0.0701	0.0472	0.1078	0.0594	0.0216	0.0162	0.0269	0.0180	0.0037	0.0024	0.0054	0.0030
ILM-IT-UI(unseen)	**0.0717**	**0.0481**	**0.1086**	**0.0600**	0.0213	0.0162	0.0269	0.0181	0.0038	0.0024	**0.0062**	**0.0032**

We also add a **ILM-rand** baseline, which is an ILM model with random initialized Q-Former encoder. For each dataset, we select the checkpoint with the best NDCG@10 metric on the development set. For all models, we use the same 8 layer transformer model as the LLM backbone. As can be seen in Table 3, our method consistently outperforms other baselines across all datasets. **ILM** also outperforms **ILM-rand**, suggesting the importance of the item-language representation learning phase. We report mean and standard error of our ILM results in Table 7.

5 Ablation Study

5.1 Effects of Q-Former Phase 1 Training.

As shown in Table 1 and Table 3, **ILM** consistently outperforms **ILM-rand** by a noticeable margin across all metrics on all benchmarks, which suggests the importance of the Q-Former phase 1 training. For the OpenP5 dataset, we experiment with different combinations of phase 1 training losses: (1) Only using item-text losses (**ILM-IT**). (2) Combine item-text losses with an item-item contrastive loss (**ILM-IT-II**). (3) Combine item-text losses with an user-item contrastive loss (**ILM-IT-UI**). We generate item-item pair data for (2) as follows. For each user, we treat two consecutive items in the history sequence as a positive pair, then we perform deduplication to get all unique pairs as the item-item pair data. The number of pairs generated are shown in Table 2.

The results for the above models are shown in Table 4. We observe that for ML1M dataset, introducing user-item or item-item contrastive losses can in general lead to performance gains, while for Beauty and Clothing there are no obvious gains. We hypothesize this is due to ML1M's item-text pair data being much scarcer and user interactions are much richer than the other two datasets. As can be seen in Table 2, comparing with other datasets, the ML1M dataset contains much fewer users and items, but much more user-item interactions. This supports our hypothesis, and suggests exploring user-interaction signals in the phase 1 representation learning can be beneficial for datasets like ML1M. To demonstrate the regularization effects of the item-item and user-item contrastive losses, we showed the phase 1 final train and eval item-grounded text generation losses in Figure 5. We can observe that adding item-item or user-item contrastive losses in phase 1 indeed can help to reduce the eval loss and close the train-eval gap.

Table 5: Effects of phase 1 item-item and user-item contrastive losses on OpenP5 phase 1 final train and eval item-grounded text generation losses.

Methods	ML1M		Beauty		Clothing	
	Train	Eval	Train	Eval	Train	Eval
ILM-IT	0.0000	4.1699	1.0441	4.2643	0.2114	2.0530
ILM-IT-II	0.1552	3.8675	2.0232	3.2567	0.4594	1.7338
ILM-IT-UI	0.0089	4.0663	2.3420	3.3724	0.5498	1.6358

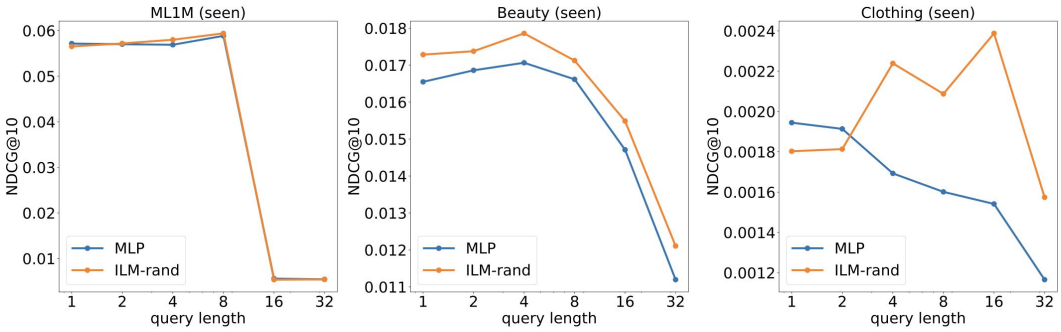

Figure 3: Effects of Number of Query Tokens.

5.2 Effects of Number of Query Tokens

Another key aspect of our ILM approach is we used multiple learned queries to generate multiple embeddings in Q-Former output as item representation to feed into LLM. Existing methods [49, 59] typically use one embedding as the item-representation to feed into LLM. We show ILM results using different numbers of queries tokens in Figure 3. In order to better understand the gains of our approach, we also use the MLP approach to project the input embedding into a same number of embeddings. For both approaches, as the number of query tokens increases, the performance first increases then decreases. For most of the query lengths, our method outperforms the MLP approach.

6 Conclusion

We propose ILM, a novel approach for incorporating collaborative filtering knowledge into a frozen LLM for conversational recommendation tasks. Specifically, we use a Q-Former item encoder to produce item-language aligned representations given item collaborative filtering embeddings, then interleave those representations with text token embeddings and feed into a frozen LLM. We propose a two-phase training paradigm. In Phase 1, we pretrain the Q-Former encoder using item-text representation learning objectives and a novel item-item representation learning objective. The item-item representation learning plays a regularization role to prevent the Q-Former from overfitting on item-text data if they are scarce. At the same time, it also directly encodes item-item similarity information in the Q-Former output representations. In Phase 2, we train ILM on conversational recommendation tasks by freezing LLM backbone parameters. This preserves the original pretrained LLM's language abilities, and reduces the privacy risk when finetuning LLM on user behavioral data. We conduct extensive benchmarks across various conversational recommendation datasets and tasks. We show the performance improvement due to item-item and user-item interaction data, demonstrating how this technique incorporates traditional CF signals. We demonstrate the value of this technique for both recommendation tasks and for language generation tasks for conversational recommendation. The results show that our model can achieve SOTA performance consistently comparing existing methods of integrating collaborative filtering knowledge into a frozen LLM.

References

[1] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, R. Ring, E. Rutherford, S. Cabi, T. Han, Z. Gong, S. Samangooei, M. Monteiro, J. L. Menick, S. Borgeaud, A. Brock, A. Nematzadeh, S. Sharifzadeh, M. a. Bińkowski, R. Barreira, O. Vinyals, A. Zisserman, and K. Simonyan. Flamingo: a visual language model for few-shot learning. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 23716–23736. Curran Associates, Inc., 2022.

[2] J. Anderson, Q. Huang, W. Krichene, S. Rendle, and L. Zhang. Superbloom: Bloom filter meets transformer, 2020.

[3] K. Bao, J. Zhang, Y. Zhang, W. Wang, F. Feng, and X. He. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*, RecSys '23. ACM, Sept. 2023.

[4] N. Belkin, C. Cool, A. Stein, and U. Thiel. Cases, scripts, and information-seeking strategies: On the design of interactive information retrieval systems. *Expert Systems With Applications*, 9(3):379–395, 1995.

[5] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020.

[6] X. Chen, X. Wang, S. Changpinyo, A. Piergiovanni, P. Padlewski, D. Salz, S. Goodman, A. Grycner, B. Mustafa, L. Beyer, A. Kolesnikov, J. Puigcerver, N. Ding, K. Rong, H. Akbari, G. Mishra, L. Xue, A. Thapliyal, J. Bradbury, W. Kuo, M. Seyedhosseini, C. Jia, B. K. Ayan, C. Riquelme, A. Steiner, A. Angelova, X. Zhai, N. Houlsby, and R. Soricut. Pali: A jointly-scaled multilingual language-image model, 2023.

[7] A. Chowdhery, S. Narang, J. Devlin, et al. Palm: Scaling language modeling with pathways, 2022.

[8] W. B. Croft and R. H. Thompson. I3r: A new approach to the design of document retrieval systems. *J. Am. Soc. Inf. Sci.*, 38:389–404, 1987.

[9] S. Dai, N. Shao, H. Zhao, W. Yu, Z. Si, C. Xu, Z. Sun, X. Zhang, and J. Xu. Uncovering chatgpt's capabilities in recommender systems. In *Proceedings of the 17th ACM Conference on Recommender Systems*, RecSys '23, page 1126–1132, New York, NY, USA, 2023. Association for Computing Machinery.

[10] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence. Palm-e: An embodied multimodal language model. In *arXiv preprint arXiv:2303.03378*, 2023.

[11] J. Fang, S. Gao, P. Ren, X. Chen, S. Verberne, and Z. Ren. A multi-agent conversational recommender system, 2024.

[12] Y. Feng, S. Liu, Z. Xue, Q. Cai, L. Hu, P. Jiang, K. Gai, and F. Sun. A large language model enhanced conversational recommender system, 2023.

[13] Y. Gao, T. Sheng, Y. Xiang, Y. Xiong, H. Wang, and J. Zhang. Chat-rec: Towards interactive and explainable llms-augmented recommender system, 2023.

[14] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang. Retrieval-augmented generation for large language models: A survey, 2024.

[15] G. Gemini Team. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024.

[16] G. Gemini Team. Gemini: A family of highly capable multimodal models, 2024.

[17] S. Geng, S. Liu, Z. Fu, Y. Ge, and Y. Zhang. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM Conference on Recommender Systems*, RecSys '22, page 299–315, New York, NY, USA, 2022. Association for Computing Machinery.

[18] M. H. Göker and C. A. Thompson. The adaptive place advisor: A conversational recommendation system, 2000.

[19] Google. Palm 2 technical report, 2023.

[20] F. M. Harper and J. A. Konstan. The movielens datasets: History and context. *ACM Trans. Interact. Intell. Syst.*, 5(4), dec 2015.

[21] R. He and J. McAuley. Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. In *Proceedings of the 25th International Conference on World Wide Web*, WWW '16, page 507–517, Republic and Canton of Geneva, CHE, 2016. International World Wide Web Conferences Steering Committee.

[22] T. Hosking, P. Blunsom, and M. Bartolo. Human feedback is not gold standard, 2024.

[23] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[24] Y. Hu, Y. Koren, and C. Volinsky. Collaborative filtering for implicit feedback datasets. In *2008 Eighth IEEE International Conference on Data Mining*, pages 263–272, 2008.

[25] Z. Hu, Y. Zhang, M. Xiao, W. Wang, F. Feng, and X. He. Exact and efficient unlearning for large language model-based recommendation, 2024.

[26] W. Hua, S. Xu, Y. Ge, and Y. Zhang. How to index item ids for recommendation foundation models. *SIGIR-AP*, 2023.

[27] A. Jaegle, F. Gimeno, A. Brock, O. Vinyals, A. Zisserman, and J. Carreira. Perceiver: General perception with iterative attention. In M. Meila and T. Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 4651–4664. PMLR, 18–24 Jul 2021.

[28] W.-C. Kang, J. Ni, N. Mehta, M. Sathiamoorthy, L. Hong, E. Chi, and D. Z. Cheng. Do llms understand user preferences? evaluating llms on user rating prediction, 2023.

[29] J. Y. Koh, D. Fried, and R. Salakhutdinov. Generating images with multimodal language models, 2023.

[30] Y. Koren, R. Bell, and C. Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.

[31] W. Kuo, A. Piergiovanni, D. Kim, X. Luo, B. Caine, W. Li, A. Ogale, L. Zhou, A. Dai, Z. Chen, C. Cui, and A. Angelova. Mammut: A simple architecture for joint learning for multimodal tasks, 2023.

[32] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020.

[33] J. Li, D. Li, S. Savarese, and S. Hoi. Blip-2: bootstrapping language-image pre-training with frozen image encoders and large language models. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.

[34] Y. Liu, W. Zhang, Y. Chen, Y. Zhang, H. Bai, F. Feng, H. Cui, Y. Li, and W. Che. Conversational recommender system and large language model are made for each other in E-commerce pre-sales dialogue. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9587–9605, Singapore, Dec. 2023. Association for Computational Linguistics.

[35] J. McAuley, C. Targett, Q. Shi, and A. van den Hengel. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '15, page 43–52, New York, NY, USA, 2015. Association for Computing Machinery.

[36] I. Mohanty. Recommendation systems in the era of llms. In *Proceedings of the 15th Annual Meeting of the Forum for Information Retrieval Evaluation*, FIRE '23, page 142–144, New York, NY, USA, 2024. Association for Computing Machinery.

[37] J. Ni, G. Hernandez Abrego, N. Constant, J. Ma, K. Hall, D. Cer, and Y. Yang. Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models. In S. Muresan, P. Nakov, and A. Villavicencio, editors, *Findings of the Association for Computational Linguistics: ACL 2022*, pages 1864–1874, Dublin, Ireland, May 2022. Association for Computational Linguistics.

[38] L. Ning, L. Liu, J. Wu, N. Wu, D. Berlowitz, S. Prakash, B. Green, S. O'Banion, and J. Xie. User-llm: Efficient llm contextualization with user embeddings, 2024.

[39] OpenAI. Gpt-4 technical report, 2024.

[40] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, UIST '23, New York, NY, USA, 2023. Association for Computing Machinery.

[41] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023.

[42] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In M. Meila and T. Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763. PMLR, 18–24 Jul 2021.

[43] S. Rajput, N. Mehta, A. Singh, R. H. Keshavan, T. Vu, L. Heldt, L. Hong, Y. Tay, V. Q. Tran, J. Samost, M. Kula, E. H. Chi, and M. Sathiamoorthy. Recommender systems with generative retrieval. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[44] S. Rendle, W. Krichene, L. Zhang, and Y. Koren. Revisiting the performance of ials on item recommendation benchmarks. In *Proceedings of the 16th ACM Conference on Recommender Systems*, RecSys '22, page 427–435, New York, NY, USA, 2022. Association for Computing Machinery.

[45] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 68539–68551. Curran Associates, Inc., 2023.

[46] N. Shazeer and M. Stern. Adafactor: Adaptive learning rates with sublinear memory cost, 2018.

[47] A. Singh, T. Vu, R. Keshavan, N. Mehta, X. Yi, L. Hong, L. Heldt, L. Wei, E. Chi, and M. Sathiamoorthy. Better generalization with semantic ids: A case study in ranking for recommendations, 2023.

[48] W. Sun, L. Yan, Z. Chen, S. Wang, H. Zhu, P. Ren, Z. Chen, D. Yin, M. Rijke, and Z. Ren. Learning to tokenize for generative retrieval. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 46345–46361. Curran Associates, Inc., 2023.

[49] G. Tennenholtz, Y. Chow, C. Hsu, J. Jeong, L. Shani, A. Tulepbergenov, D. Ramachandran, M. Mladenov, and C. Boutilier. Demystifying embedding spaces using large language models. In *The Twelfth International Conference on Learning Representations*, 2024.

[50] M. Tsimpoukelli, J. L. Menick, S. Cabi, S. M. A. Eslami, O. Vinyals, and F. Hill. Multimodal few-shot learning with frozen language models. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 200–212. Curran Associates, Inc., 2021.

[51] J. Wang, Y. Liang, F. Meng, Z. Sun, H. Shi, Z. Li, J. Xu, J. Qu, and J. Zhou. Is ChatGPT a good NLG evaluator? a preliminary study. In Y. Dong, W. Xiao, L. Wang, F. Liu, and G. Carenini, editors, *Proceedings of the 4th New Frontiers in Summarization Workshop*, pages 1–11, Singapore, Dec. 2023. Association for Computational Linguistics.

[52] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen. A survey on large language model based autonomous agents. In *Front. Comput. Sci.*, 2024.

[53] W. Wang, Q. Lv, W. Yu, W. Hong, J. Qi, Y. Wang, J. Ji, Z. Yang, L. Zhao, X. Song, J. Xu, B. Xu, J. Li, Y. Dong, M. Ding, and J. Tang. Cogvlm: Visual expert for pretrained language models, 2024.

[54] L. Wu, Z. Zheng, Z. Qiu, H. Wang, H. Gu, T. Shen, C. Qin, C. Zhu, H. Zhu, Q. Liu, H. Xiong, and E. Chen. A survey on large language models for recommendation, 2023.

[55] S. Xu, W. Hua, and Y. Zhang. Openp5: Benchmarking foundation models for recommendation. *arXiv:2306.11134*, 2023.

[56] J. Yu, Z. Wang, V. Vasudevan, L. Yeung, M. Seyedhosseini, and Y. Wu. Coca: Contrastive captioners are image-text foundation models, 2022.

[57] J. Yu, Y. Xu, J. Y. Koh, T. Luong, G. Baid, Z. Wang, V. Vasudevan, A. Ku, Y. Yang, B. K. Ayan, B. Hutchinson, W. Han, Z. Parekh, X. Li, H. Zhang, J. Baldridge, and Y. Wu. Scaling autoregressive models for content-rich text-to-image generation, 2022.

[58] Y. Zhang, H. Ding, Z. Shui, Y. Ma, J. Zou, A. Deoras, and H. Wang. Language models as recommender systems: Evaluations and limitations. In *NeurIPS 2021 Workshop on I (Still) Can't Believe It's Not Better*, 2021.

[59] Y. Zhang, F. Feng, J. Zhang, K. Bao, Q. Wang, and X. He. Collm: Integrating collaborative embeddings into large language models for recommendation, 2023.

Table 6: The mean and standard error of the ILM metrics for ELM 24 tasks (computed using 3 runs with different random seeds)

Tasks	SC(%)	Log pplx
summary	0.7318 ± 0.00359	0.6250 ± 0.00134
positive review	0.7832 ± 0.00246	0.5711 ± 0.00211
neutral review	0.7680 ± 0.00584	0.5759 ± 0.00345
five pos char.	0.8631 ± 0.00320	0.5960 ± 0.00556
five neg char.	0.9225 ± 0.00017	0.4029 ± 0.00845
long description	0.7258 ± 0.00411	0.6622 ± 0.00130
funnier	0.6891 ± 0.00358	0.5394 ± 0.00085
sadder	0.7181 ± 0.00267	0.5152 ± 0.00112
scarier	0.7241 ± 0.00261	0.5251 ± 0.00104
improve	0.7759 ± 0.00293	0.5123 ± 0.00264
movie to viewer	0.8008 ± 0.00313	0.6005 ± 0.00425
pitch	0.8370 ± 0.00341	0.5181 ± 0.00198
criticize	0.7894 ± 0.00356	0.5504 ± 0.00406
convince1	0.7881 ± 0.00353	0.6176 ± 0.00353
convince2	0.7957 ± 0.00523	0.7795 ± 0.00533
convince3	0.7949 ± 0.00421	0.8900 ± 0.00714
dissuade1	0.7909 ± 0.00203	0.6076 ± 0.00381
dissuade2	0.8386 ± 0.00167	0.7278 ± 0.00609
similarities	0.8422 ± 0.00585	0.3993 ± 0.00842
interpolation	0.7300 ± 0.00363	0.5310 ± 0.00238
why like nn	0.8030 ± 0.00367	0.6452 ± 0.00556
diff than nn	0.8891 ± 0.00198	0.5111 ± 0.00941
common with nn	0.8433 ± 0.00182	0.5393 ± 0.00817
all	0.7939 ± 0.00317	0.5813 ± 0.00415

Table 7: The mean and standard error of the ILM metrics for OpenP5 datasets (computed using 3 runs with different random seeds)

Datasets	HR@5	NDCG@5	HR@10	NDCG@10
ml1m(seen)	0.0715 ± 0.00051	0.0476 ± 0.00047	0.1072 ± 0.00043	0.0591 ± 0.00020
beauty(seen)	0.0210 ± 0.00018	0.0160 ± 0.00020	0.0265 ± 0.00036	0.0177 ± 0.00025
clothing(seen)	0.0040 ± 0.000085	0.0025 ± 0.000014	0.0061 ± 0.00022	0.0032 ± 0.000051
ml1m(unseen)	0.0720 ± 0.00044	0.0478 ± 0.00052	0.1069 ± 0.00095	0.0591 ± 0.00049
beauty(unseen)	0.0214 ± 0.00019	0.0162 ± 0.000090	0.0266 ± 0.00020	0.0179 ± 0.000099
clothing(unseen)	0.0038 ± 0.000037	0.0025 ± 0.000041	0.0059 ± 0.00015	0.0032 ± 0.000029

A Preserving Pretrained Ability

In conversational recommendation, often the user and the system will conduct multiple turns of conversations, and for the system to achieve a certain goal, tool use may be employed [12, 13, 34, 11]. There is often no constraint on the topics of the conversations, so the pretrained abilities of the LLM could be important. For example, the user may ask the LLM to perform certain operations such as adding or removing certain items from the recommended items, or conducting certain filtering based on a criteria. Those operations require certain reasoning ability from LLM pretraining. For another example, to use LLMs as an automatic agents [52, 40] for conversational recommendation, they may require certain broad knowledge to perform tool use [45] to achieve a task. If the LLM is later fully finetuned only using the task specific data, it is likely that those pretrained abilities will be lost or hidden. Our ILM approach uses a frozen LLM, when the inputs don't contain items, the behavior of the model will be exactly the same as the original LLM. This means all pretrained knowledge can be preserved, which is crucial for multi-turn conversations and tool use in automatic agents.

B Hyperparameters

The hyper-parameters for phase 1 and phase 2 trainings on the ELM 24 tasks are presented in Table 8.

Table 8: Hyper-parameters for ELM 24 tasks.

Phase 1	Q-Former	8 layers, 168M params
	batch size	256
	learning rate	3×10^{-5}
	learning rate schedule	cosine decay
	optimizer	AdaFactor [46]
	# steps	259K
	hardware	16 Cloud V5 TPUs
Phase 2	LLM backbone	PaLM 2-S [19]
	batch size	32
	learning rate	5×10^{-4}
	learning rate schedule	linear decay
	optimizer	AdaFactor [46]
	# steps	100K
	hardware	64 Cloud V5 TPUs

For OpenP5 benchmarks, we use a 8 layer transformer decoder as the LLM backbone. We pretrain the LLM backbone using text only OpenP5 data to enable generative retrieval. For the ML1M dataset, we pretrain for 100K steps. For the Beauty dataset, we pretrain for 20K steps. For the Clothing dataset, we pretrain for 10K steps. The hyper-parameters for phase 1 and phase 2 trainings on the OpenP5 tasks are presented in Table 9.

Table 9: Hyper-parameters for OpenP5 tasks.

Phase 1	Q-Former	8 layers, 168M params
	batch size	256
	learning rate	3×10^{-5}
	learning rate schedule	cosine decay
	optimizer	AdaFactor [46]
	# steps	40K for ML1M, 10K for Beauty, 15K for Clothing
	hardware	16 Cloud V5 TPUs
Phase 2	LLM backbone	Transformer decoder 8 layers, 128M params
	batch size	32
	learning rate	5×10^{-4}
	learning rate schedule	linear decay
	optimizer	AdaFactor [46]
	# steps	50K for ML1M, 20K for Beauty, 20K for Clothing
	hardware	64 Cloud V5 TPUs